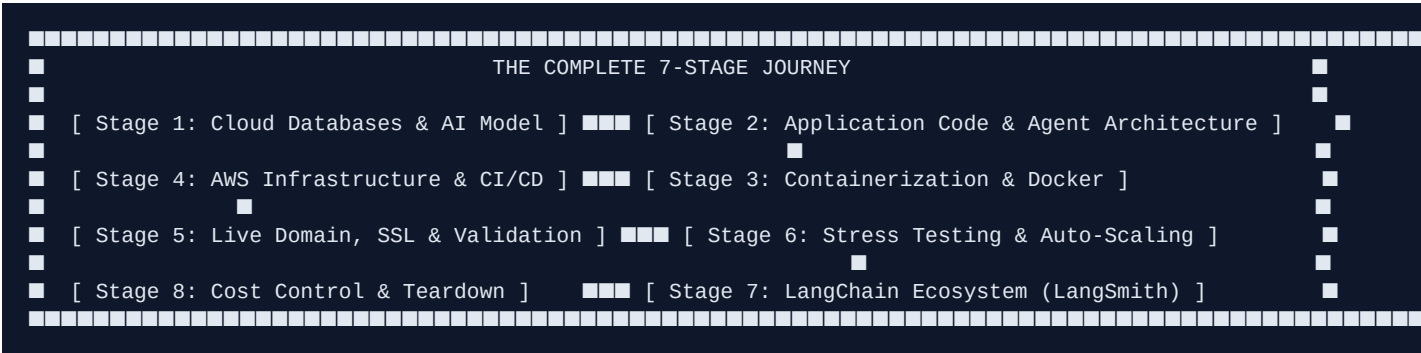


Maison Luxé (STELLA) — Master End-to-End Project Checklist & Roadmap

Complete Lifecycle Checklist: From Database Provisioning to Production Cloud Deployment & LangSmith Observability

■ Project Phases at a Glance



■■ Stage 1: Cloud Databases & AI Model Setup

✓ 1.1 Provision Aiven Cloud MySQL Database (Transactional Data)

- Created managed MySQL service on Aiven Cloud (Port 16512 over TLS).
- Seeded luxury fashion tables: `products` , `inventory` , `orders` , and `categories` .
- Configured database credentials & verified public host connectivity.

✓ 1.2 Provision Supabase PostgreSQL Database (Agent Memory & Checkpoints)

- Created PostgreSQL instance on Supabase.
- Enabled the **IPv4 Connection Pooler** (Port 5432 on `aws-0-*.pooler.supabase.com`).
- Configured LangGraph / LangChain checkpoint schema for thread-safe conversation memory persistence.

✓ 1.3 SenseNova LLM API Account Setup

- Generated SenseNova API Key for natural language understanding, SQL query generation, and luxury fashion styling recommendations.

■ Stage 2: Application Code & Agent Architecture

✓ 2.1 Backend Microservice (FastAPI & Python 3.12)

- Built `mysql_db.py` : MySQL database connection manager and safe query executor.
- Built `supabase.py` : PostgreSQL connection pooler for session checkpoint persistence.
- Built `auth.py` : JWT authentication with HMAC-SHA256 signing, supporting 3 distinct demo profiles:
 - **Priya Sharma** (Platinum VIP Tier)
 - **Aisha Khan** (Gold Tier)
 - **Riya Verma** (Gold Tier)
- Built `agent.py` : LangChain ReAct agent integrating SQL tools with SenseNova LLM.
- Built `main.py` : REST API exposing `/health` , `/products` , `/auth/users` , and authenticated `/chat` endpoints.

✓ 2.2 Frontend Client (Luxury Single Page App)

- Built interactive, mobile-responsive UI (`index.html` , `style.css` , `app.js`).
- Implemented multi-user profile switcher (dynamically attaches JWT Bearer token).
- Configured `nginx.conf` for reverse proxying, HTTP caching, and gzip compression.

✓ 2.3 Unit Testing & Code Quality

- Configured `pytest` unit test suite covering health checks, auth validation, and database mock responses.
- Enforced `ruff` linting and formatting standards.

■ Stage 3: Containerization & Docker Engineering

✓ 3.1 Backend Dockerfile

- Multi-stage build with Python 3.12 slim base image.
- Runs with non-root security privileges on port `8000` .

✓ 3.2 Frontend Dockerfile

- Lightweight Nginx Alpine base image (~25 MB).
- Configured static asset serving on port `80` .

■■ Stage 4: AWS Infrastructure & GitHub Actions CI/CD

✓ 4.1 Networking & Firewalls (VPC & Security Groups)

- Created `alb-sg` : Public internet firewall (Inbound HTTP 80 & HTTPS 443).
- Created `ecs-tasks-sg` : Container firewall (Inbound 80 & 8000 strictly restricted to `alb-sg`).

✓ 4.2 Container Registries (Amazon ECR)

- Created private ECR repositories: `fashion-backend` and `fashion-frontend` .
- Enabled vulnerability scan-on-push.

✓ 4.3 Secrets Management (AWS SSM Parameter Store)

- Encrypted all sensitive variables under `/fashion-agent/*` (`MYSQL_PASSWORD` , `POSTGRES_PASSWORD` , `SENSENOVA_API_KEY` , etc.).

✓ 4.4 Centralized Observability (CloudWatch Logs)

- Created log groups `/ecs/fashion-backend` and `/ecs/fashion-frontend` with 30-day retention policies.

✓ 4.5 IAM Roles & Security

- Created `ecsTaskExecutionRole` with inline permissions to decrypt SSM secrets, pull ECR images, and stream CloudWatch logs.
- Created IAM user `github-actions-deployer` with `AmazonEC2ContainerRegistryPowerUser` and `AmazonECS_FullAccess` .

✓ 4.6 Serverless Compute (Amazon ECS Fargate)

- Created ECS cluster `fashion-agent-cluster` .
- Registered task definitions: `deploy/ecs-backend-task.json` and `deploy/ecs-frontend-task.json` .
- Created `fashion-backend-service` and `fashion-frontend-service` .

✓ 4.7 Automated CI/CD Pipeline (GitHub Actions)

- Configured `.github/workflows/deploy.yml` with secrets (`AWS_ACCESS_KEY_ID` , `AWS_SECRET_ACCESS_KEY` , `AWS_REGION`).
- Automated testing, Docker building, ECR pushing, and zero-downtime rolling deployments on `git push origin main` .

■ Stage 5: Custom Domain, SSL & Live Routing

✓ 5.1 DNS Delegation (GoDaddy \$→ Route 53)

- Created Route 53 Public Hosted Zone for `predictoraa.com` .
- Updated GoDaddy nameservers with AWS NS records (`awsdns-*`).

✓ 5.2 Wildcard SSL Certificate (AWS ACM)

- Issued free public certificate for `predictoraa.com` and `*.predictoraa.com` via Route 53 DNS validation.

✓ 5.3 Application Load Balancer (ALB)

- Created `alb-fashion-agent` .
- Attached Port 80 listener (HTTP \$→ HTTPS 301 Permanent Redirect).
- Attached Port 443 HTTPS listener with ACM Certificate:
- Path Rule: `/chat*` , `/products*` , `/health*` , `/auth*` \$→ Forward to `tg-fashion-backend` (Port 8000).
- Default Rule: `/*` \$→ Forward to `tg-fashion-frontend` (Port 80).

✓ 5.4 Apex & Subdomain Alias Mapping

- Created Route 53 A (Alias) records pointing `predictoraa.com` and `www.predictoraa.com` to the ALB.

■ Stage 6: Performance Testing & Auto-Scaling

✓ 6.1 Postman Load Testing Collection

- Created test suite with Bearer token authentication for `/health` , `/products` , and `/chat` .
- Configured Postman **Performance Runner** with virtual user ramp-up profiles.

✓ 6.2 Automated Python Multi-Threaded Load Tester

- Built `scripts/stress_test.py` with multi-threading, dynamic JWT retrieval, and optional `.env` overrides.

✓ 6.3 ECS Application Auto-Scaling

- Attached Target Tracking Scaling Policy targeting **70% Average CPU Utilization**.
- Configured minimum 1 task, maximum 5 tasks, 60s scale-out cooldown, and 300s scale-in cooldown.

✓ 6.4 Real-Time Monitoring & Verification

- Monitored real-time request logs using **CloudWatch Start tailing (Live Tail)**.
- Verified container CPU/Memory utilization graphs and ECS scale-up/scale-down lifecycle events.

■ Stage 7: Connecting to the LangChain Ecosystem (LangSmith)

■ The Final Production Milestone:

Connecting your deployed STELLA agent to **LangSmith** provides full enterprise LLM observability, trace visualizers, token cost analytics, and prompt debugging.

Why Connect LangSmith?

1. **Trace Visualizer:** View the full execution tree of every prompt (SenseNova LLM call $\rightarrow$ SQL tool invocation $\rightarrow$ Result synthesis).
2. **Latency & Cost Tracking:** Accurately monitor token counts, pricing per request, and latency bottlenecks in real time.
3. **Debugging Bad Queries:** Inspect exact generated SQL queries and LLM hallucinations directly from the web dashboard.

How to Integrate LangSmith (Step-by-Step):

7.1 Create a LangSmith Account & API Key

1. Go to https://smith.langchain.com and log in.
2. Go to **Settings** $\rightarrow$ **API Keys** $\rightarrow$ Click **Create API Key**.
3. Copy your key: `lsv2_pt_...`

7.2 Store LangSmith Secrets in AWS SSM Parameter Store

Run this command in your terminal:

```
`bash
aws ssm put-parameter \
--name "/fashion-agent/LANGCHAIN_API_KEY" \
--value "lsv2_pt_YOUR_LANGSMITH_KEY" \
--type "SecureString" \
--overwrite \
--region ap-south-1
`
```

7.3 Add LangSmith Environment Variables to Backend Task Definition

In `deploy/ecs-backend-task.json`, add these variables under the container definition:

```
`json
"environment": [
{ "name": "LANGCHAIN_TRACING_V2", "value": "true" },
{ "name": "LANGCHAIN_ENDPOINT", "value": "https://api.smith.langchain.com" },
{ "name": "LANGCHAIN_PROJECT", "value": "maison-luxe-stella" }
],
"secrets": [
{
"name": "LANGCHAIN_API_KEY",
"valueFrom": "/fashion-agent/LANGCHAIN_API_KEY"
}
]
`
```

■ 7.4 Push Changes to Deploy via GitHub Actions

```
`bash
git add .
git commit -m "feat: enable LangSmith enterprise LLM tracing in production"
git push origin main
`
```

■ 7.5 View Live Traces in LangSmith Dashboard

Open [\[https://smith.langchain.com\]](https://smith.langchain.com)(<https://smith.langchain.com>) $\rightarrow$ Open project `maison-luxe-stella` to watch live trace graphs for every customer interaction!

■ Stage 8: Teardown & Cost Management

✓ 8.1 Documented Complete Resource Teardown Runbook

- Created [\[docs/aws_teardown_and_cleanup_guide.md\]](#)([file:///Users/sachinmishra/Desktop/sql_fashion_brand_agent/docs/aws_teardown_and_cleanup_guide.md](#)) with 1-click cleanup script and UI steps to guarantee a **1\$0.00 AWS bill**.